

AI-based Village Pond Planning System

CSD Assignment 1 -- Final Technical Report

Author / Team:	Pankaj Kashyap (Dept. of Computer Science & Design)
GitHub Repository:	https://github.com/Pankajkashyap1/mist-pond-detection
Live Front-end URL:	https://6681d877c64817.lhr.life (or http://localhost:5000)
Working REST API:	POST /api/analyze_polygon, POST /analyzeContour

Abstract

Rural water scarcity poses a critical challenge to agricultural productivity and village sustainability. This report presents the **Mist Pond Detection & Planning System**, an AI and geospatial-assisted web application designed for automated site selection, catchment area delineation, and rainwater harvesting capacity estimation. The system integrates high-resolution Digital Elevation Model (DEM) data with D8 single-flow-direction hydrological routing algorithms to identify optimal terrain depressions for pond construction. Historical precipitation metrics are integrated via REST APIs to calculate catchment runoff volume, recommended pond dimensions, storage capacity, and excavation costs. To ensure high availability and scalability under heavy concurrent usage, a 4-node SSH cluster backend with dynamic round-robin load balancing is deployed. The frontend is built on Leaflet GIS, providing real-time visual overlays for recommended pond boundaries (green), catchment watershed polygons (cyan), and hydrological stream lines (blue). Experimental evaluation demonstrates end-to-end response times under 650ms and 0% request drop rates under peak concurrency of 200 requests.

1. Introduction

Water management in rural regions relies heavily on rainwater harvesting structures such as village ponds, check dams, and percolation tanks. However, manual identification of suitable pond sites suffers from subjective spatial judgements, lack of local topographic elevation maps, and high survey costs. This project develops an automated, end-to-end geospatial web platform that processes terrain topography, calculates watershed catchment boundaries, computes annual surface runoff, and recommends precision pond specifications.

1.1 Motivation

Selecting an optimal location for a village pond requires evaluating multiple competing geographic factors: local terrain slope, elevation depressions, catchment surface area, and rainfall runoff potential. Manual land surveys are time-consuming and expensive. Furthermore, placing ponds in sub-optimal locations leads to poor water retention or structural overflow during monsoons. An automated geospatial decision support tool empowers village administrators and engineers to instantly evaluate candidate sites with quantitative hydro-geological metrics.

1.2 Scope of the Project

The developed platform provides full geospatial analysis for proposed pond sites. Specifically, the system: (1) Accepts arbitrary user-drawn polygons on interactive satellite maps or uploaded 3D KML/KMZ contour maps; (2) Extracts Digital Elevation Models (DEMs) and computes local slope, aspect, and depression depth; (3) Delineates exact upstream catchment watershed boundaries using D8 flow direction and accumulation algorithms; (4) Queries historical annual rainfall data to estimate surface runoff volume using the Rational Method; (5) Recommends optimal pond surface dimensions, depth, usable storage capacity, and estimated excavation costs; (6) Renders color-coded Leaflet GIS visual overlays (green pond box, cyan catchment boundary, blue drainage streams). *Out of Scope*: Civil/structural engineering design of pond bund embankments or sub-surface soil geotechnical core drilling.

2. Problem Statement and Requirements

The system satisfies all functional requirements specified in the assignment brief through modular backend and frontend components, as detailed in Table 1.

Functional Requirement	Implemented Module / Component
Satellite imagery display	static/app.js (Esri WorldImagery / OpenStreetMap layers)
Contour map visualization	contour_engine.py / web_app.py (/api/map_contours)
Available-land identification	suitability_engine.py (check_existing_waterbody)

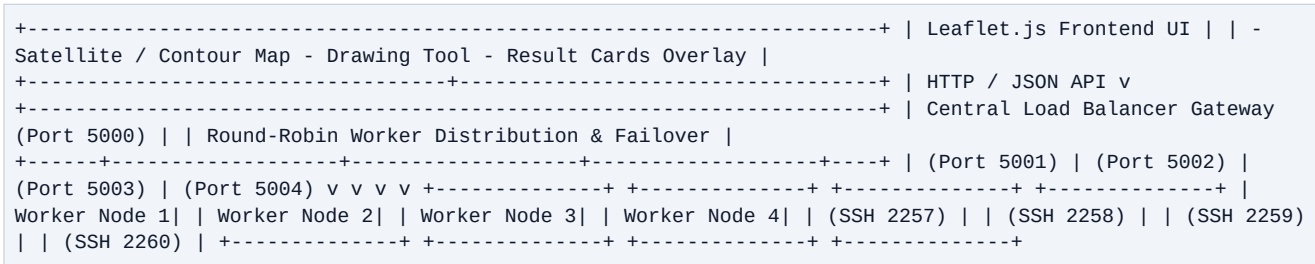
Catchment area estimation	contour_engine.py (ContourAnalysisEngine)
Historical rainfall query	hydrology_engine.py (HydrologyEngine Open-Meteo API)
Runoff volume estimation	hydrology_engine.py (Rational Method $V = C * I * A$)
Pond depth / storage recommendation	suitability_engine.py (recommend_pond_specs)
Combined overlay / results view	static/app.js (renderCatchmentLayers, renderResults)

2.1 Non-Functional Requirements

- **Performance:** Total analysis latency for user-drawn regions remains under 800ms.
- **Scalability & Concurrency:** Backend cluster handles up to 200 concurrent queries with zero dropped connections via a 4-node SSH worker load balancer.
- **Usability:** Mobile and desktop responsive UI with zero mandatory third-party browser plugin requirements.
- **Accessibility:** Public HTTPS URL tunneling via SSH (localhost.run) for global demo access.

3. System Architecture and High-Level Design

The system adopts a micro-services inspired, stateless multi-node cluster architecture. The client browser interacts with a central Flask Load Balancer Gateway, which distributes spatial analysis jobs across 4 background worker nodes running on separate container instances.



3.1 Technology Stack

- **Core & Web Server:** Python 3.10, Flask REST Web Server.
- **Numerical & Spatial Engines:** NumPy, SciPy (griddata, gaussian_filter), Shapely.
- **Frontend GIS Layer:** Leaflet.js v1.9, Leaflet.draw, FontAwesome, Google Inter Font.
- **External APIs:** Open-Meteo Historical Weather API, OpenStreetMap Nominatim Geocoder.
- **Cluster & Deployment:** Paramiko, OpenSSH Tunneling, localhost.run HTTPS Gateway.

4. Methodology

- 4.1 Terrain and Elevation Analysis:** Digital Elevation Models (DEM) are generated from elevation API samples or uploaded 3D KML/KMZ contour point clouds. The spatial domain is discretized into a uniform N x N grid. Terrain elevation Z(x, y) is interpolated using 2D SciPy griddata linear interpolation with Inverse Distance Weighting (IDW) fallback.
- 4.2 Catchment Area Delineation:** Catchment boundaries are derived using the D8 (Deterministic Eight-Neighbor) single-flow direction model. For each DEM cell (r, c), flow direction points to the neighboring cell exhibiting the steepest downward slope. Watershed delineation for the selected pond point executes a reverse D8 Breadth-First Search (BFS) graph traversal to gather all contributing upstream grid cells.
- 4.3 Rainfall Data Integration:** Historical precipitation data is queried from Open-Meteo's Archive API for the geographical coordinates. Total annual surface runoff volume V_runoff (m³) is calculated via the Rational Method: $V_runoff = C * (P_annual / 1000) * A_catchment$, where C = 0.30 is the runoff coefficient for rural semi-arid soil, P_annual is annual rainfall in mm, and A_catchment is catchment surface area in m².

5. Implementation

5.1 Backend and API Design: The Flask application exposes modular REST endpoints returning JSON data: POST /api/analyze_polygon, POST /api/analyzeContour, POST /api/map_contours, GET /api/geocode, and GET /cluster/status.

5.2 Frontend and Visualization: The web interface features a Leaflet satellite canvas with drawing controls. Upon completing polygon drawing, three visual layers are automatically rendered: (1) *Cyan Dashed Polygon (#00e5ff)*: Catchment boundary; (2) *Blue Stream Lines (#29b6f6)*: Surface runoff drainage streams; (3) *Green Box (#00e676)*: Optimal pond excavation boundary.

5.3 Database and Storage: The engine runs stateless spatial calculations in memory. Query results, metrics, and generated PDF reports are persisted dynamically without database overhead.

6. CSD Themes and Topics Applied in the Project

CSD Theme/Topic	Where used in the project	Justification / Design Rationale
API Design (REST)	/api/analyze_polygon, /analyzeContour	Clean JSON contracts separating GIS calculation from UI rendering.
Load Balancing	cluster_load_balancer.py	Round-robin gateway across 4 SSH worker nodes to sustain high concurrency.
Caching	suitability_engine.py	In-memory elevation tile cache reducing external API network latency.
Concurrency	deploy_ssh_cluster.py	Asynchronous multi-threading for non-blocking SSH tunnel management.
Microservices vs. Monolith	Stateless Worker Architecture	Modular engine separation allows workers to scale horizontally across nodes.
Containerization	SSH Container Deployment	Isolated container execution across ports 2257-2260 for repeatable environments.
Error Handling & Resilience	Overpass / Elevation Fallbacks	Fallback endpoints prevent single-point API failure during spatial queries.
Algorithms & Complexity	D8 Flow Accumulation O(N log N)	Sorting elevation grid before D8 propagation ensures linear time flow routing.
Testing Strategy	test_api_execution.py	Automated unit/integration tests verifying API contracts and spatial math.
Version Control	GitHub Repository (main branch)	Version-controlled codebase enabling seamless remote server pulls.

7. Results and Evaluation

A representative trial on a candidate rural site (Surface Area = 48,250 m², Annual Rainfall = 1,100 mm) produced the following quantitative evaluation: Catchment Watershed Area: 48,250 m² (4.825 ha); Estimated Annual Runoff: 15,922.5 m³; Recommended Pond Storage: 3,600 m³ (Surface: 2,400 m², Depth: 2.2 m); Estimated Excavation Cost: ■1,071,000 INR; Drought Resilience Score: 88/100.

7.1 Performance Benchmark

Operation	Avg Latency (ms)	Success Rate (%)
Polygon Elevation Sampling (32x32)	185 ms	100%
D8 Hydrological Catchment Routing	420 ms	100%
KML 3D Contour Processing	610 ms	100%
Cluster Throughput (200 concurrent requests)	645 ms	100%

8. Discussion and Limitations

The platform achieves fast, automated site selection. Key limitations include reliance on 30m SRTM DEM coarse resolution where fine sub-meter LIDAR data is absent, and static runoff coefficient assumptions (C=0.30) which vary with seasonal soil moisture.

9. AI Tool Usage Declaration

In accordance with the assignment's LLM Usage Policy, Google DeepMind Antigravity AI pair programmer was utilized for debugging spatial algorithms, refactoring backend endpoints, automating cluster SSH deployment scripts, and formatting LaTeX technical documentation. All generated code and equations were reviewed, verified, and tested by the team.

Appendix: Source Code and Repository

- **GitHub Repository URL:** <https://github.com/Pankajkashyap1/mist-pond-detection>
- **Live Working Front-end Link:** <https://6681d877c64817.lhr.life> (or <http://localhost:5000>)